

Add and organise learning resources

1. What you will make and how to use this guide

You will build My first offline lesson: a small plant lesson that opens on any school device through the portal. Learners press a button to reveal an answer. You then add an optional picture, PDF, audio and video. Finally you can create a separate media shelf with Python. All example code is fresh; you do not need another portal's files.

Your route through the workbook

- Pages 2-5: create folders and three text files; test a working lesson. No Python or media download is needed.
- Pages 6-7: create or obtain optional media, record its source and add local links/players.
- Page 8: register the HTML entry in the main portal catalogue. Python 3 is needed for catalogue rebuilding.
- Pages 9-10: optional Python exercise that builds a separate list and browser media shelf.

Know the file types before you start

HTML (.html) contains headings, paragraphs, links and page structure. CSS (.css) sets colours and layout. JavaScript (.js) runs in a learner's browser and responds to clicks. JSON (.json) stores structured data. Python (.py) runs on the server only when you ask it to rebuild a list. The portal is static: it does not run Python when a learner opens a page.

You need

- The portal already installed and reachable. These examples assume C:\LearningPortal and HTTP port 80. Replace the path or append your actual port where needed.
- File Explorer, Notepad and a browser on the server. Word or another editor is useful only to create an optional PDF.
- Permission to edit the installed web folder. If Windows refuses a save, use the authorised server administrator account.
- A dated backup of C:\LearningPortal\web before edits.

A faster route is included

The ZIP contains examples/lesson-starter with complete files. You may copy that whole folder as described on page 2. To learn how to develop your own resources, follow the manual creation steps instead. The examples do not need internet libraries, fonts or login services.

2. Create folders and learn to save text files

Step 1 - open the live resource folder

Press Windows+E. Click the address bar, paste `C:\LearningPortal\web\resources` and press Enter. You should see Portal guides. Right-click an empty area > New > Folder and name it Activities. Open Activities, create a folder named lesson-starter, and open it. Inside it create another folder named media. Do not create these inside runtime or vendor.

```
C:\LearningPortal\web\resources\Activities\lesson-starter
C:\LearningPortal\web\resources\Activities\lesson-starter\media
```

Step 2 - show the real filenames

In Windows 11 choose View > Show > File name extensions. In Windows 10 use the View tab > File name extensions. You must be able to see .html, .css and .js. A file named index.html.txt is a text file with the wrong name and will not become the lesson homepage.

Step 3 - use this save procedure for every code file

- Open Start, type Notepad and open it. Paste ONLY the code; do not paste the surrounding explanation or the triple backticks from Markdown.
- Choose File > Save As (often Ctrl+Shift+S). Browse to the lesson-starter folder above.
- Set File name to the exact name stated on the page. Set Save as type to All Files (*.*) and Encoding to UTF-8. Click Save.
- Return to Explorer and check the final extension. To edit later, right-click the file > Open with > Notepad, then use Ctrl+S.
- Do not use Word to save HTML, CSS or Python code: it may add formatting. Close/reopen Notepad files to confirm your work was saved.

Alternative - copy the ready-made folder

In the extracted portal ZIP open examples. Copy lesson-starter. Open the installed resources\Activities folder and paste it. Avoid nesting lesson-starter inside another lesson-starter folder. If an old practice folder already exists, back it up before replacing it.

Check before continuing

The lesson-starter folder is inside the installed web\resources\Activities folder, and media is inside lesson-starter. Pages 3, 4 and 5 will add index.html, style.css and lesson.js beside each other. File names and directory names in a URL must match your actual files.

3. Create the lesson page with HTML

Step 1 - save the complete code as index.html

Use the Notepad procedure on page 2. Save in lesson-starter, not in media. The name index.html lets NGINX open this file when a URL ends with the folder name.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My first offline lesson</title>
  <link rel="stylesheet" href="style.css">
  <script src="lesson.js" defer></script>
</head>
<body>
  <main>
    <h1>My first offline lesson</h1>
    <p>Plants need water, light and air to grow.</p>
    <h2>Try a question</h2>
    <p>Which part of a plant absorbs water?</p>
    <button id="show-answer" type="button">Show answer</button>
    <p id="answer" hidden>Roots absorb water from the soil.</p>
    <!-- Add optional media here later. -->
  </main>
</body>
</html>
```

Step 2 - understand what you can change

Change the title, h1, lesson paragraph, question and answer to suit a subject. Keep the IDs show-answer and answer: JavaScript uses these names. The hidden attribute keeps the answer invisible until the button is used. style.css and lesson.js are relative paths to files in this same folder.

Step 3 - first browser check

On the server open <http://localhost/resources/Activities/lesson-starter/> . The text may look plain and the button will not yet work because the next two files are not created. A 404 means the folder/file path is wrong. Do not include C:\LearningPortal or /web/ in the browser address.

4. Add appearance with CSS

Step 1 - save as style.css beside index.html

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 1rem;
  background: #f1f5f9;
  color: #18324b;
}
main {
  max-width: 48rem;
  margin: auto;
  padding: 1.5rem;
  background: white;
}
h1 { color: #11685c; }
button { font-size: 1rem; padding: .7rem; cursor: pointer; }
button:focus { outline: 3px solid #b36b00; }
img, video { max-width: 100%; height: auto; }
audio { max-width: 100%; }
a { color: #005a9c; }
```

Step 2 - refresh and look for the change

Save the file. Return to the HTTP lesson address and press Ctrl+F5. Expected: a light background, white reading area, green title and padded button. If nothing changes, inspect the filename: style.css.txt is wrong. Also check the HTML says href="style.css" exactly.

Step 3 - practise one safe edit

In Notepad change max-width: 48rem to max-width: 40rem, save and refresh. The reading area becomes narrower on a wide screen. Change it back if desired. Values after # are colour codes; a semicolon ends a declaration. Keep the braces around each group.

What the selectors mean

body styles the whole page. main styles the central reading area. h1 styles the main heading. img and video keep media from becoming wider than the page. button:focus shows where a keyboard user is working. Avoid removing the focus outline or choosing text/background colours that are hard to distinguish.

Expected result

The lesson is now readable on a phone and desktop. The button still needs JavaScript on page 5. This stylesheet affects only this lesson; editing it does not change the main portal style.css. You can reuse this lesson style for other activity folders.

5. Make the answer button work with JavaScript

Step 1 - save as lesson.js beside index.html

```
const button = document.querySelector("#show-answer");
const answer = document.querySelector("#answer");
button.addEventListener("click", () => {
  answer.hidden = !answer.hidden;
  button.textContent = answer.hidden
    ? "Show answer"
    : "Hide answer";
});
```

Step 2 - test the interaction

- Save lesson.js. Open or refresh the lesson through its HTTP URL.
- Click Show answer: Roots absorb water from the soil should appear and the button should read Hide answer.
- Click again: the answer disappears. Test using Tab to focus the button and Enter to activate it.
- Open the same lesson on a learner device using the server IP, for example <http://192.168.1.25/resources/Activities/lesson-starter/> . Replace that example IP.

Understand the code a line at a time

querySelector finds the HTML element with the named ID; # means an ID. The variables button and answer hold those elements. addEventListener runs the function when the button is clicked. The ! sign reverses the hidden value. textContent changes the displayed button label, choosing one label when hidden and another when visible.

Step 3 - adapt the question without changing the code

Open index.html in Notepad. Replace the question with What does evaporation mean? and the answer with Liquid water changes into water vapour. Save and refresh. Leave the IDs unchanged. This creates a different lesson without rewriting lesson.js.

If the button fails

- Check lesson.js is in the same folder as index.html, and is not named lesson.js.txt.
- Check the HTML script line uses src="lesson.js" and defer.
- Check the IDs are spelled show-answer and answer in both files.
- Restore the ready-made lesson.js if you accidentally removed a quote, bracket or parenthesis. The examples folder is your reference copy.

You now have a complete local activity. No media download, Python or internet connection is required for this core lesson.

6. Create or obtain optional PDF, image, audio and video

Start with your own small files

These four practice names go inside lesson-starter\media: worksheet.pdf, plant-photo.jpg, explanation.wav and plant-video.mp4. They are illustrative filenames, not included media. Use only the ones you need. A lesson can work with a PDF alone or with no media at all.

- PDF: write a short worksheet in Word or another document editor. Choose File > Print, printer Microsoft Print to PDF, then save worksheet.pdf into media. An editor's Export as PDF is another option. Open the PDF to check it.
- Image: take your own plant photograph, transfer it to the server and copy a genuine JPEG into media as plant-photo.jpg. Do not turn a PNG into JPEG merely by renaming its extension.
- Audio: record a short explanation with a phone or recording app, then transfer the actual WAV file. If your recording is MP3 or M4A, keep its real extension and update the HTML on page 7.
- Video: record a brief demonstration and transfer a genuine MP4 file. Use a modest resolution and test the actual learner browser.

Download reusable media from Wikimedia Commons

Open <https://commons.wikimedia.org/> while online. Search for a suitable plant image, audio or video. Open the individual FILE description page, read its licence and author, then follow the original-file download link. Save the downloaded file into media and retain its genuine format. Use a compatible file or change the page-7 paths to match. A Commons web page saved as .html is not the media file.

For a video you uploaded to YouTube

Sign in to YouTube Studio at <https://studio.youtube.com/> . Select Content, find your own video, open its menu and choose Download. Copy the resulting MP4 into media and name it plant-video.mp4. A Premium offline download stays within YouTube and is not a reusable portal MP4. For another creator's video, obtain an authorised downloadable file from that creator; do not assume any public YouTube video can be copied.

Keep a source record

Use Notepad to create media/credits.txt. For each file record its title, creator, source URL, licence/permission and download date. A CC BY file needs attribution; follow the actual licence conditions. The MIT licence for this portal code does not grant rights to downloaded content. Sources: Commons reuse guide and YouTube download help are listed on page 10.

7. Place media in the lesson and check every path

Step 1 - insert only the tags for files you have

Open index.html with Notepad. Locate `<!-- Add optional media here later. -->` inside main. Replace that comment with the relevant code below, before `</main>`. Keep the rest of the original page. These paths assume files inside the media subfolder.

```
<h2>Look at a plant</h2>

<h2>Read and write</h2>
<p><a href="media/worksheet.pdf">Open the worksheet PDF</a></p>
<h2>Listen</h2>
<audio controls preload="none" src="media/explanation.wav">
  <a href="media/explanation.wav">Download audio</a>
</audio>
<h2>Watch</h2>
<video controls preload="none" src="media/plant-video.mp4">
  <a href="media/plant-video.mp4">Download video</a>
</video>
<p>Media credits: <a href="media/credits.txt">Sources and licences</a></p>
```

Step 2 - understand relative paths

media/worksheet.pdf means: from the HTML page's folder, enter media and open worksheet.pdf. For a new file called seed-growth.mp4, change both occurrences of media/plant-video.mp4 to media/seed-growth.mp4. URLs use /, not Windows backslashes. Avoid filenames copied from unrelated examples.

Step 3 - verify before sharing

- Save index.html and refresh. Click the PDF link, inspect the picture and play audio/video. Missing files usually produce a 404 or a broken player.
- If you saved audio as explanation.mp3, change the source and download link to .mp3. Renaming the extension does not convert the recording.
- If a file exists but will not play, test another genuine supported format or browser. Preserve the original source; do not assume the portal converts codecs.
- Add a transcript paragraph or linked transcript.txt; write meaningful alt text for the image. Supply captions if you have a captioned video.
- Disconnect internet, keep Wi-Fi/LAN connected and retest from a learner phone. Fully local resources should continue working.

Avoid embedding the online YouTube player for an offline lesson: it still depends on internet access. Large videos can consume LAN bandwidth; test real classroom use rather than promising a fixed learner count.

8. Register the lesson and rebuild the main catalogue

Step 1 - create index.html.meta.json

In Notepad save the complete JSON below beside index.html, using All Files and UTF-8. This is an HTML metadata sidecar: it tells the main catalogue to list this entry page. It must not be named index.meta.json or index.html.meta.json.txt.

```
{
  "title": "My first offline lesson",
  "category": "Activities",
  "language": "English",
  "source": "Tirtharaj Dhungana - original starter example",
  "rights": "MIT for starter code; record added content rights separately",
  "description": "A local plant lesson with a show-answer button."
}
```

Use double quotes around every field and text value. Keep commas between fields but not after the last one. Edit the title and description to match your lesson; replace source/rights appropriately for your own work. Keep source/permission records for each added media file.

Step 2 - install Python 3 once on the server

While online, open <https://www.python.org/downloads/windows/>. Choose a supported stable Python 3 Windows installer that matches the computer: normally 64-bit on an x64 server. Do not select the embeddable ZIP for this exercise. Use the installer's normal installation process; if it offers Add Python to PATH, enable it. Finish installation, then close and reopen PowerShell. Check one of these commands:

```
py -3 --version
python --version
```

At least one must report Python 3. If neither works, check the installation/launcher and open a new terminal. Do not download an unrelated program because a command is missing. Browsing the portal does not need Python; generating the catalogue does.

Step 3 - run the installed rebuild

& C:\LearningPortal\Rebuild_Catalog.bat

Run this in PowerShell, or double-click the installed BAT in Explorer. Read the result and press a key if asked. Refresh the portal home page, search My first offline lesson and open its HTML card. The main generator lists PDF/audio/video automatically but only lists HTML entries with sidecars. Support files such as CSS and JS are not catalogue cards.

If rebuilding reports an error

Read the named file and message. Fix invalid JSON or wrong text values, then retry. The previous catalogue is preserved on generation errors. Do not hand-edit catalog.json to work around the error: the next rebuild replaces it.

9. Optional: generate a media list with Python

Your lesson and main catalogue already work. This extra exercise teaches how a local script can produce data for another HTML page. It creates media-list.json for the files directly inside this lesson's media folder. It does not replace or modify the main catalogue generator.

Step 1 - save as build_media_list.py in lesson-starter

```
from pathlib import Path
from urllib.parse import quote
import json

folder = Path(__file__).resolve().parent
media = folder / "media"
allowed = {".pdf", ".mp3", ".wav", ".mp4", ".webm"}
items = []
for file in sorted(media.iterdir()):
    if file.is_file() and file.suffix.lower() in allowed:
        items.append({
            "title": file.stem.replace("-", " "),
            "url": "media/" + quote(file.name)
        })
(folder / "media-list.json").write_text(
    json.dumps(items, indent=2, ensure_ascii=False),
    encoding="utf-8")
print(f"Listed {len(items)} media files")
```

Step 2 - run the script in PowerShell

Press Start, type PowerShell and open it. Paste these two lines in order. The cd command changes the working folder; the script also locates itself using `__file__`, so it writes into its own lesson folder.

```
cd C:\LearningPortal\web\resources\Activities\lesson-starter
py -3 build_media_list.py
```

If only python works on page 8, use `python build_media_list.py` instead. Expected: Listed N media files. If there are no PDF/audio/video files in media, N is 0; this is a valid empty result. Images and credits.txt are intentionally excluded.

Step 3 - inspect what Python wrote

Open media-list.json with Notepad. Each item has a title and a relative URL. The script turns worksheet.pdf into title worksheet and URL media/worksheet.pdf. Spaces in names are URL-encoded. It only scans the immediate media folder; do not put practice files in another nested folder.

What to change later

Add or remove media, then rerun this script. Python does not run automatically when you change a file. A `PermissionError` means you cannot write the target; use the authorised account. A missing media directory means you saved the script in the wrong place or forgot page 2. On page 10 you will display this generated list.

10. Create the optional media shelf

Step 1 - save library.html beside index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>My media shelf</title>
<link rel="stylesheet" href="style.css">
</head>
<body><main>
<h1>My media shelf</h1>
<p><a href="index.html">Back to lesson</a></p>
<ul id="list"></ul>
<p id="status" role="status">Loading...</p>
</main><script src="shelf.js"></script></body>
</html>
```

Step 2 - save shelf.js in the same folder

```
const list = document.querySelector("#list");
const status = document.querySelector("#status");
fetch("media-list.json").then(r => {
  if (!r.ok) throw new Error("Missing list");
  return r.json();
}).then(items => {
  for (const item of items) {
    const row = document.createElement("li");
    const link = document.createElement("a");
    link.href = item.url;
    link.textContent = item.title;
    row.append(link);
    list.append(row);
  }
  status.textContent = `${items.length} files available`;
}).catch(() => {
  status.textContent = "Run build_media_list.py and reload.";
});
```

fetch reads the JSON made on page 9. Each item becomes a local link; textContent displays its title as text. Open <http://localhost/resources/Activities/lesson-starter/library.html> . Expect N files available. Do not double-click the file: use HTTP for fetch.

Add `<p><a href="library.html">Open my media shelf</a></p>` before `</main>` in index.html. To create another topic, copy the whole lesson folder, change HTML and metadata, replace media/credits, rerun Python if used and rebuild the main catalogue. Test from a learner device.

References checked 3 October 2026: <https://support.google.com/youtube/answer/56100> ;
https://commons.wikimedia.org/wiki/Commons:First_steps/Reuse ;
<https://www.python.org/downloads/windows/> . Complete fresh code is in examples/lesson-starter.